Observer-Indexed Distinction Calculus over Transform Programs

Scoped tests, retained witnesses, control and certified abstraction

Mettapedia Working Notes

8 November 2026

Abstract

An abstraction is justified relative to its tests, permitted interactions and preservation claim. Distinction calculus supplies such observations of a transform program without restricting the ambient language to a finite or metric model. We formalize scoped predicate-test certificates and their transport, contextual readings, independently failing witness axes, open client interactions, bounded control guarantees, consumer-sensitive error bounds, and probabilistic mass preservation. Exact state quotients and a terminating finite minimizer are certified instances of this broader account. Table machines and the serial distinction-history grammar lower to transform with two-way correspondence. The development retains the limits of each claim: finite tests need not characterize infinitely branching behavior, approximate quotients need not have a greatest element, and a serial interpreter does not establish true-concurrency preservation.

1 The ambient system and its observation contracts

The primary object is a program with its executions and retained evidence. An observer selects tests and readings of that object; it does not replace the object's entire structure by a scalar. Different observers can protect ordered results, probabilities, causal histories, costs or authority. Their preservation laws need not be identical or even symmetric.

The implementation reuses the existing observation discipline

$$\text{event history} \longrightarrow \text{retained witnesses} \longrightarrow \text{semantic reading} \longrightarrow \text{optional policy score}.$$

Collection, chronological composition, independent composition and ordering are separate capabilities. No shared semiring or quantale is imposed on every witness container. The exact quotient developed next is one useful profile.

The MeTTaDinC specification organizes models around states, protected queries, actions, representations and evidence-bearing requests [1]. A transform program already supplies states and admitted labelled firings. Let

$$s \xrightarrow{a} t \quad \Longleftrightarrow \quad \exists f,\ \text{offered}(a, s, f) \wedge \text{admitted}(f) \wedge t = \text{result}(f).$$

This uses the actual transform admission law, including resources, information and guards; it is not an unrelated transition model. Attach a protected reading $r : S \to V$. A tuple of queries is a single such reading; when query values have a metric tolerance, V may be their protected zero-distance quotient.

An encoding $q : S \to Q$ is a candidate compression. Define its labelled successor-code set by

$$N_q(s, a) = \{q(t) \mid s \xrightarrow{a} t\}.$$

The exact obligations are

$$q(s) = q(t) \Longrightarrow r(s) = r(t),$$
$$q(s) = q(t) \Longrightarrow N_q(s, a) = N_q(t, a) \quad \text{for every } a.$$

The second condition preserves availability as well as nondeterministic alternatives. It makes no probabilistic claim about their weights.

Theorem 1 (Exact observer)**.** *The two obligations hold exactly when* $\ker q$ *is an observation-preserving bisimulation of the labelled transform program.*

Lean: `DistinctionObserver.exact_iff_kernel_bisimulation`. Consequently equal codes satisfy the same Hennessy–Milner experiments, including nested branching experiments, not just the same current queries (`exact_preserves_modal`). This preservation direction does not require finite branching.

This is the appropriate connection between the two calculi. An encoding of *states* is not automatically a morphism of *substitution frames*. Constraint-solver licenses concern solution families; observer adequacy concerns transitions and observations. Further representation theorems may connect them, but the names alone do not identify their types or obligations.

2 Scoped tests and changing observers

Instead of only naming a literal reading value, an observer may ask a predicate of that value. For $r : S \to V$, its tests are $p : V \to \mathsf{Prop}$, interpreted as $p(r(s))$. This gives the same bisimulation contract as literal value readings, but makes coarsening compositional: a test p of $g \circ r$ translates to $p \circ g$ on r. The encoding need not enumerate these predicates to prove the semantic law.

Theorem 2 (Scoped certificate transport)**.** *Index the actual observed systems by a scope* i*, and let a certificate at* i *prove that* $\ker q$ *is a bisimulation of that system. A proved observer refinement from coarse tests and labels to fine ones transports a certificate from the fine scope to the coarse scope. A status label or matching revision number alone does not provide this transport.*

Lean: `tests_bisimulation_iff`, `readingRefinement`, `Certificate`, `certificate_forget`. The scope indexes the semantic system itself. A native cache must additionally implement the correspondence between its stored identity and that semantic subject; these theorems do not assume a cryptographic digest proves semantic preservation. This certificate transport fixes the underlying carrier and equation theory. Transport across different operational theories additionally needs the appropriate interpretation or compilation-adequacy theorem.

Weakening protected tests while keeping the encoding fixed is different from merging additional codes. The latter changes the encoding kernel and needs a new compatibility proof. The eight-state control `merging_exact_codes_can_fail` has an exact AB encoding whose projection to A is not update-compatible, even though the protected query remains A.

A concrete control uses an inert two-bit machine with self-loops. Encoding the first bit is exact when only that bit is read. Changing the protected query to the second bit invalidates the encoding, even though the dynamics are unchanged. The development proves both the old certificate and refusal of the new one, and that no observer-refinement witness licenses the invalid transport (`changed_query_refuses_old_encoding`, `no_refinement_to_new_query`).

Proposition 1 (The boundary of unrestricted exact compression)**.** *An encoding preserves every semantic state predicate exactly if and only if it is injective. If it merges states with different values of a newly requested reading, no decoder of that code alone can recover the reading.*

Lean: `all_predicates_iff_injective`, `lost_reading_has_no_decoder`. These propositions do not assert that all predicates are computable. They show why an evolving observer needs retained information or re-observation, rather than an unqualified promise of permanent universal compression.

3 Contexts and open interaction

For a declared monoid of contexts C acting on states, saturate a reading by

$$r_C(s)(c) = r(c \cdot s).$$

Equality of these readings is automatically closed under further contexts: composition of contexts reindexes the tests. The identity context recovers the original reading, and every context-closed relation protecting r is contained in $\ker r_C$. If an exact encoding also has a context-closed kernel, its certificate upgrades to these contextual readings. Lean: `contextRead_closed`, `contextRead_universal`, `exact_contextRead`. In the bit-swap control, saturation of a first-bit query retains both bits. This monoid construction is one capability; partial or dependent interactions are not required to be total context functions.

For open operational systems, program and client labels are disjoint labels on firings of the *same* transform step. Client firings can be partial, guarded and state-dependent. Stability under their union is exactly stability under each family (`DistinctionOpen.stable_iff`). The resulting exact certificate covers arbitrary interleavings of the declared client and program moves. A control certifies the first-bit view for a program that only idles, then refutes it when a client can swap the two bits (`closed_exact_open_refused`). This states the environment contract instead of assuming that closed-program equivalence survives every client.

4 Witnesses and independent observation axes

The transform-run adapter feeds the existing observation-discipline and evaluator-control interfaces. A history observer can deliberately ignore the endpoints; an endpoint-bearing view keeps them explicitly, including for an empty run. A retained full history may be read again without replaying effects.

For an indexed family of partial collectors D_i, retain

$$\big(D_i.\mathit{collect}(h)\big)_i \quad \text{and read} \quad \big(D_i.\mathit{observe}(h)\big)_i.$$

Each coordinate has its own optional result. An absent collection is not a logical refutation; detailed statuses or failure causes belong in the axis's declared result type. One absent coordinate does not erase the value or witnesses of another. Equality of the indexed result is exactly equality of every coordinate (`indexed_eq_iff`); no totality assumption is required. The control `independent_failure` distinguishes this from a strict product that returns one joint failure. Another control has equal length scores but different retained histories.

These observations reuse the evaluator's existing preservation and pruning contract. The theorem `control_preserves_runs` connects a licensed history change to the observations of actual runs. It does not grant transition, pruning or publication authority merely because a score improved. A finite event word also does not by itself encode an infinite execution; progress and divergence require their own state or execution observations.

5 Quotients without representative switching

Define the existential quotient transition

$$c \xRightarrow{a} d \quad \iff \quad \exists s, t,\ q(s) = c \wedge s \xrightarrow{a} t \wedge q(t) = d.$$

For an unstable encoding this is an over-approximation: a subsequent abstract step can silently use a different representative of the same code.

Theorem 3 (Lifting from every representative)**.** *If the update obligation holds, then for every s, a, d,*

$$q(s) \xRightarrow{a} d \quad \iff \quad \exists t,\ s \xrightarrow{a} t \wedge q(t) = d.$$

Lean: `DistinctionObserver.quotient_lifts`. Surjectivity is obtained by taking the code carrier to be the image of q; unused declared codes need not be populated by invented states.

There are two different optimization questions. Refining a supplied encoding must retain its present distinctions, whereas minimizing for a query family need only retain the protected queries. The refinement algorithm below accepts an arbitrary seed relation, so the distinction is in the seed, not in a second execution engine.

6 A terminating certified finite implementation

For finite state and action types, an elaborated machine consists of a reading and successor lists:

$$r : S \to V, \qquad \mathit{next} : S \to A \to \mathrm{List}(S).$$

Here lists are read as finite successor sets. Empty lists encode unavailable actions. Multiplicity and probability require richer observations.

For a finite relation $R \subseteq S \times S$, remove a pair unless its current readings agree and every successor on each side has a related successor on the other. Write the resulting subset as $F(R)$. Repeatedly replace R by $F(R)$ until equality. Every nonfinal round strictly decreases $|R|$, so this procedure terminates without an assumed search budget.

Theorem 4 (Greatest stable refinement)**.** *The implementation returns a stable subset of its input containing every stable subset of that input. Starting from* $\ker r$*, its result is precisely observation-preserving bisimilarity, hence an equivalence relation and the coarsest exact behavioural quotient protecting r.*

Lean: `DistinctionFinite.Machine.normalize`, `behaviour_greatest`, `mem_behaviour_iff`, and `behaviouralSetoid`. This is an executable reference algorithm, not a claim of optimal implementation complexity. Efficient partition refinement can replace it under the same semantic specification.

Eight-state calibration. For $s = (a, b, c) \in \{0, 1\}^3$, read a and act by swapping a, b. Keeping only a fails: $(0, 0, 0)$ and $(0, 1, 0)$ initially agree but their next readings differ. Keeping (a, b) succeeds, and the computed partition has exactly four classes. The third bit remains present in the concrete state while being irrelevant to the declared interface.

Access-policy calibration. In the specification's access example, merging the three pre-lockout stages gives a valid nondeterministic simulation but not an exact quotient. The abstract failure transition can lock out immediately, while the first concrete stage cannot. The formal controls establish both simulation and this extra behavior. A safe over-approximation remains useful, but it cannot be promoted to exact executable compression.

7 Actual transform execution

The machine compiler represents state by (n, s), with one current occurrence named n. An action-successor pair (a, t) is a firing that consumes n and makes $n + 1$. Its admission is exactly $t \in next(s, a)$, and its result is $(n + 1, t)$. The clock supplies fresh occurrence identities and is absent from the protected reading.

Theorem 5 (Two-way machine lowering)**.**

$$(n, s) \xrightarrow{a} (m, t) \quad \Longleftrightarrow \quad m = n + 1 \wedge t \in next(s, a).$$

Moreover, the finite encoding checker accepts exactly the generic transform observer obligation for the compiled machine.

Lean: `DistinctionFinite.Machine.moves_iff`, `checkEncoding_exact`, `exactEncoding_iff_transform`, and `checked_preserves_modal`. The compiler does not choose one member of a nondeterministic successor set. It also does not claim that a serialized machine preserves parallel independence.

Beyond finite state. The machine lowering itself does not require a finite carrier. The existing distinction-history grammar operates on multisets of live nodes using evolution, fork, merge and erasure. Its executable event function becomes the successor list of the same compiler. The formal correspondence `DistinctionHistory.run_iff` equates source execution with existence of a transform run having the same event word and final multiset, from any fresh clock. A control executes all four event forms. A second control rejects merging two copies when only one occurrence exists.

This is an unbounded-state serial interpretation. To preserve true concurrency, compile live nodes to separate occurrences and prove that the chosen event independence is preserved. That is a stronger backend theorem, not a reason to restrict the semantic history language to finite state.

8 Graded and stochastic observations

A tolerance is reflexive, symmetric and bounded; it need not be transitive. It therefore cannot in general replace the exact equality used by frames. The existing distinction library already treats metric coherence and zero-distance quotienting as additional properties.

For arbitrary bounded real-valued query families on transform states, `DistinctionObserver.gradedReadings` supplies the existing quantitative modal semantics with the actual transform transitions. Its induced tolerance is metric. When each label has finitely many distinct successor states, its logical distance equals the existing Hausdorff behavioural distance (`gradedReadings_metric`, `gradedReadings_behavioural_eq`). No finite state-space assumption is imposed here. The theorem is semantic; it is not an effective algorithm for all real-valued systems.

Stochastic actions require pushforward of mass:

$$(q_* \mu_{s,a})(c) = \sum_{q(t)=c} \mu_{s,a}(t).$$

Exact stochastic compatibility equates these masses for equal source codes. Set-valued compatibility is weaker. The transform adapter now uses the existing finitely supported subdistribution semantics directly. Sound support proves that each positive-weight outcome is an actual admitted transform transition. Complete support is a separate law: a declared policy may assign zero weight to an otherwise possible move. A concrete point-policy control is sound but not complete for the uncommitted two-alternative program.

Theorem 6 (Exact stochastic abstraction)**.** *Preservation of current readings and of the pushed-forward masses of every code event is equivalent to probabilistic bisimulation of* $\ker q$*. Preservation of whole-space mass also preserves refusal mass.*

Lean: `exact_iff_probabilistic_bisimulation`, `refusal_mass_eq`, `support_iff_move`. These theorems connect a declared law to transform transitions. They do not derive probabilities from an unweighted relation. The finitely supported law is a profile; it is not a restriction imposed on all ambient probability models.

An action label is also an input, not necessarily a random event. If an external client chooses a, sample the successors conditional on that request. Normalizing over every enabled action changes the interface by randomizing the client's choice. A pending-request occurrence is one natural transform representation of this ordering.

Approximate compression does not always have a single greatest quotient. The existing control `GradedCongruenceControls.no_greatest_decodable` exhibits incompatible approximate merges. Optimization must therefore state its objective and constraints rather than promise that every tolerance has a canonical optimal partition.

9 Control guarantees and consumer-sensitive error

The same unowned transition graph can describe choices made by the agent or by its environment. At an agent-owned state, define the certain predecessor of G by

$$\exists a,\ \mathrm{enabled}(s,a) \wedge \forall t\ (s \xrightarrow{a} t \Rightarrow G(t)).$$

At an environment-owned state require a nonempty successor family and G after every enabled action and outcome. Nonemptiness prevents deadlock from proving a vacuous success. Iterating these predecessors with the goal gives the bounded guarantee of reaching the goal within n steps.

Theorem 7 (Ownership-sensitive control preservation)**.** *A bisimulation preserving the goal and who chooses preserves every such bounded certain-reachability guarantee. In particular, an exact encoding whose protected reading includes both owner and goal preserves the guarantee.*

Lean: `DistinctionControl.force_congr`, `force_of_exact`. The actual transform control has the same ordinary branching view in two states, but an agent can ensure success in one while the environment can prevent it in the other. This is bounded, perfect-state control under demonic outcomes. It is not a theorem about stochastic winning probabilities, hidden-state strategies or unbounded fairness.

For approximate readings, let $d(r(s), \widehat{r}(s)) \le \epsilon$. An L-Lipschitz consumer f satisfies

$$d(f(r(s)), f(\widehat{r}(s))) \le L\epsilon.$$

A subsequent approximation of error η gives $L\epsilon + \eta$. These laws hold for arbitrary pseudometric value spaces; they do not require the values to be bounded similarities (`DistinctionSensitivity.Approximates.consumer`, `Approximates.comp`).

A discontinuous threshold needs a margin. For real values,

$$|x - y| \le \epsilon < |x - \theta| \quad \Longrightarrow \quad (\theta \le x \iff \theta \le y).$$

Lean: `threshold_preserved_of_margin`. The 0.49/0.51 control crosses a 0.5 threshold despite value error 0.02. Thus a grade must identify the quantity it bounds and the consumer assumptions used to transport it.

10 Limits retained as part of the theory

Several limitations cannot be repaired by deleting a hypothesis:

- An existing infinite-branching control has logical distance zero but behavioural distance one. The finite-branching hypotheses in the metric characterization remain explicit; they are not imposed on the ambient transform language.
- Perfect one-sided nonexpansion permits a constant map that loses all distinctions. Preservation of a task's readings and two-sided distortion are different claims.
- Approximate decodability need not have a greatest quotient. Optimality therefore needs its objective and admissible class, and may yield several incomparable certified candidates.
- Set support, probability mass, ordered alternatives, causal histories, completion and cost are different observations. The existing ordered, probabilistic, productive-block and causal theories supply their own laws.

The relevant existing controls are `quantitative_adequacy_needs_image_finiteness`, `Tolerance.collapse_grades`, `no_greatest_decodable`, `fair_biased_not_lsBisimilar`, `status_separates`, `step_strictly_finer` and `hp_strictly_finer`.

11 A smaller implementation architecture

The following decomposition preserves the MeTTaDinC vocabulary without requiring a second general-purpose runtime.

Language object	Operational or mathematical interpretation
Machine and action	A labelled transform program; availability is admission.
Protected query	A declared observation of states or histories.
Representation	An encoding plus its current-observation and dynamic obligations.
Exact induction	A certified quotient; every abstract transition lifts.
Refinement/minimization	Search for a stable relation; independent checking or a proved producer.
Graded defect	Distance under the declared observation and transition lifting.
Stochastic compatibility	Equality of class masses, not just matching support.
Evidence status	A projection of a scoped claim and its evidence/assumptions.
Transfer or hive query	A library construction over declared response-preserving maps; no extra primitive firing is required.

The seven result statuses are not automatically a lattice or a single confidence scale. Checked and Refuted assert opposite facts about a claim; Conditional carries assumptions; Estimated carries an estimand and a statistical guarantee; the remaining outcomes describe resolution or capability failures. Bind results to the model, interface, operation family, revision and scope before interpreting a status.

Kernel correspondences and executable transfers are different constructions in MeTTaDinC itself. A cycle of noninvertible abstraction maps need not return an automorphism. Retain a round-trip obligation, and use group-valued holonomy only when the relevant invertibility and gauge action have been established. These analyses can be programs using the common substrate rather than new foundational operations.

12 Assurance and remaining boundaries

The additions are available through the Lean import `Mettapedia.Languages.ProcessCalculi.ConstraintMultisetRewriting.Step.Distinction`. They establish the indexed contracts, their composition laws and controls, the mathematical bridge and its finite implementation. They do not implement the full MeTTaDinC parser, all registered requests, a stochastic numerical backend, or a native CeTTa/tcalc lowering.

For a concrete DSL backend, the remaining route is: elaborate the admitted surface module into the typed machine/observer data; compile operations into the selected transform representation; implement or invoke a justified analysis backend; and check evidence against the same frozen subject. The source elaborator and native executor need their own adequacy theorems. A DRAT certificate cannot be replayed merely because another proof kernel exists; its proof language, checker and claimed subject must be connected.

Checking a program's operational semantics reduces one modelling boundary. It does not eliminate the boundary between a formal program, its runtime, its external services and the physical system. MeTTaDinC already makes this distinction explicit. Retaining it makes the shared foundation stronger.

The executable finite observer layer is a restriction inside the ambient calculus. Infinite histories, residual computations and richer observations retain their meaning outside that restriction. No objective-free universal "best observer" is asserted. The design criterion is to preserve exactly the declared semantic obligations while making the justified changes of observer, representation and execution explicit.

References

[1] B. Goertzel, *MeTTaDinC: An Executable Language for Observer-Relative Abstraction, Evidence, and Composition*, specification and validation fixtures, 2026. https://drive.google.com/file/d/1zRkY6sbRLIZaoXkpp4lYRN36JG5A6O0t/view.

[2] B. Goertzel, *MeTTaDinC*, executive summary, explainer, implementation guide and worked examples, 2026. https://drive.google.com/drive/folders/1QNkx4elmZw0qUf7oQADHkZXr28ERsybW.